Preserving Human Agency in AI-Assisted Decision-Making

Tommy Nicol and **Alyssa Signorelli** and **Ruoyu Wu** and **Feyza Altunal**
University of Pittsburgh
Bias and Ethical Implications in AI

1 Problem Statement

AI can change how students engage with coursework by providing real-time feedback and personalized support. However, these capabilities can also allow students to outsource cognitive effort, especially when AI provides more assistance than requested. A student asking for a hint may want guidance while still solving the problem themselves, but AI may instead provide a full solution, depriving them of the opportunity to learn and potentially raising academic integrity concerns.

If AI is integrated into learning platforms, it must be able to calibrate its assistance to the learner's needs. This project investigates whether LLMs exceed students' requested levels of programming assistance and whether system-level prompting strategies can reduce this over-assistance while preserving response correctness and usefulness.

2 Related Work/Literature

As AI tools become more common in classrooms, recent research has started looking into how well LLMs actually help beginner programmers, and where they tend to fall short. Most of the work in this space falls into three main buckets: how reliably models handle student code, how different types of hints affect student learning, and what kind of guardrails platforms use to keep AI helpful.

The first major issue is that LLMs have a hard time sticking to the boundaries of what a student actually asks for. When Hellas et al. (Hellas et al., 2023) tested models like GPT-3.5 and Codex with real student help requests, they found that while the models were good at spotting logic mistakes, they couldn't stop themselves from leaking full code solutions even when the prompt explicitly told them not to. This shows that standard models don't have a built-in way to respect a student's preferred level of help.

Researchers are also looking at how help is structured. Xiao et al. (Xiao et al., 2024) ran a study with novice programmers to see how different types of hints affect them, ranging from text explanations down to direct code examples. They found that giving only high-level natural language hints often frustrated students who were stuck on things like basic syntax errors. Their work shows that learners need a mix of conceptual guidance and concrete code examples, and that static hint styles don't fit every situation.

To handle these issues in classroom settings, developers have started building custom platforms with strict safety measures. For example, Liffiton et al. (Liffiton et al., 2023) built CodeHelp, a web app that wraps an LLM in structural guardrails to give students 24/7 debugging support without handing over the answers. While tools like CodeHelp work well, they require building and maintaining external software infrastructure. Our project builds on this past research by looking at whether we can achieve similar results using just system-level prompt engineering, to see if careful prompt design can stop models from over-assisting students.

3 Proposed Approach

3.1 Data and Experimental Setup

We will construct 20–30 programming scenarios based on common introductory programming difficulties, sourced from online forums or created ourselves. Each scenario will include a programming problem, the student's difficulty, and a request for a hint, explanation, or complete solution. The same scenarios will be tested with the same LLM under three system-instruction conditions. We will write prompts for LLMs that follow the prompt engineering guidelines below:

- Provide clear, specific, and logically organized context. Explain the problem in simple terms and include all relevant information

before asking the LLM to respond. For example, instead of asking "Why is my code slow?" explain what the code is supposed to do, describe the performance issue, and then ask the LLM to identify possible causes and provide at least two potential solutions.
- Provide examples of desired output when appropriate. When possible, show the LLM what a good or complete response should look like. For example, "Please show me how to write the Fibonacci sequence using recursion similar to this approach: xyz." This helps the LLM understand what type of response is expected.

3.2 Evaluation

We will evaluate output by comparing how well each prompting strategy follows the level of assistance requested by the student. Since each scenario will be tested under all three conditions, we can directly compare the results. We will use three main evaluation criteria:

- Assistance-level adherence: Does the AI provide the type and amount of help requested? For example, giving a complete solution when the student asks for a hint would be considered over-assistance. We will measure the rates of adherence, over-assistance, and under-assistance.
- Technical correctness: Is the programming information provided by the AI accurate?
- Usefulness: Does the response address the student's specific difficulty and help them make progress?

We will create a rubric before evaluating the responses so that each condition is judged using the same criteria. We will then compare the results across the three prompting strategies.

4 Expected Output

The expected output is a structured dataset and comparative analysis of LLM programming assistance. Each record will include the scenario, LLM, requested assistance level, generated response, and rubric scores for adherence, correctness, and usefulness.

The primary result will be a comparison of the three prompting conditions, focusing on assistance-level adherence and over-assistance, with secondary comparisons of correctness and usefulness.

5 Ethical Considerations

One ethical issue is that our project is focused on AI giving students too much assistance, which could affect how much of the work students do themselves. An AI that provides a complete solution when a student only asks for a hint could reduce the student's opportunity to learn and could also create academic integrity concerns.

There are also privacy and data concerns with the sources we use. We will only use publicly available data or examples that we create ourselves, and we will avoid including personal or identifying information from online posts. Another concern is that our evaluation of usefulness and acceptable assistance will involve human judgment, so our results could be affected by our own biases. We will try to reduce this by creating the evaluation criteria before looking at the results.

6 Project Plan and Contributions

6.1 Proposed Steps

1. Review relevant research and finalize the literature review.
2. Create 20–30 coding scenarios and define acceptable assistance levels.
3. Develop prompts and test each scenario under the three system instructions.
4. Collect and evaluate responses using the established rubric.
5. Compare results, analyze limitations, and complete the final report.

6.2 Group Member Contributions

The roles below represent each member's primary responsibilities, but are not fixed. All group members will contribute to and support each other's tasks throughout the project.

Code Experiment Infrastructure Tommy will develop the experiment infrastructure, collect AI responses, and organize the results.

Prompt Engineering Alyssa will design and refine the prompts and ensure consistency across scenarios and conditions.

Literature Review and Research Ruoyu will review research on AI assistance, student learning, agency, and AI alignment.

Ethical Analysis and Writing Feyza will analyze ethical issues and contribute to result analysis and final writing.

References

Arto Hellas, Juho Leinonen, Sami Sarsa, Charles Koutcheme, Lilja Kujanpää, and Juha Sorva. 2023. Exploring the responses of large language models to beginner programmers' help requests. *arXiv preprint arXiv:2306.05715*.

Mark Liffiton, Brad Sheese, Jaromir Savelka, and Paul Denny. 2023. Codehelp: Using large language models with guardrails for scalable support in programming classes. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*.

Ruiwei Xiao, Xinying Hou, and John Stamper. 2024. Exploring how multiple levels of gpt-generated programming hints support or disappoint novices. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pages 1–10.